
Numerical Analysis, 2026, Lab 1

Deadlines for submissions:

First attempt: February 22nd

Resit: 5 working days after the first discussion

Keywords: Interpolation, numerical integration

Instructions

Before you start

- All programming exercises must be completed using Python with JuPyteR notebooks.
- Make sure that you have done the Tutorial before starting the programming assignments. You are expected to work on it in this order: Lecture A → Tutorial A (@home) → Programming A (@lab session); then, the same with parts B, etc.
- Solutions will NOT be published afterwards. You can ask for feedback to the TAs during each scheduled session.
- Students HAVE to work in pairs and are expected to collaborate with each other. DO NOT divide tasks, that carries the (high) risk of failing the tests, as labs are the primary mode of learning in the course.
- Please give us your feedback during and/or right after the discussion with the TA. **THIS IS THE LINK TO THE FEEDBACK FORM.**

Submission

- The deadline written above indicates when (latest) to upload your JuPyteR notebook (a single one, containing all answers) to the Assignments' page in Brightspace. Both students of the pair need to upload it. If this is not the case, the "not uploading student(s)" will receive the minimal grade. They can still run the discussion in the schedule slot, but it will count as RESIT for that student, so no additional opportunity will be given.
- The failure to meet the submission deadline will result in a failing grade for the lab and therefore, the course. No resit opportunity will be given in this case.

Discussion

- The TA will contact all students in the large group by email to propose time slots for the discussion. It is sufficient for one member of the pair to book a discussion slot; the partner only needs to attend the discussion.
- If neither student in a pair replies within 48 hours of receiving the email from the TA, that pair will be considered as having missed the discussion and will receive the minimal grade for this lab. No resit opportunity will be given in this case.
- If the proposed times conflict with other **teaching** activities, both students should clearly indicate this to the TA in their reply so that an alternative can be arranged.

- At the discussion, the theoretical answers and code should be readily accessible at the start of the session. During the discussion, the students will demonstrate their programs, show the derivations in the theoretical assignments, and respond to any questions.

Grading

- All group partners will be expected to participate in all aspects of the discussion. The same grade will be given to all students.
- In the event of a failed lab discussion at the initial attempt (i.e., grade below 5.5), there is an opportunity to improve your grade within a deadline specified in the lab instructions, which you need to schedule with your TA at the very same moment of the first discussion.
- Repairs are only allowed if the group can show a serious attempt of solving ALL questions at the first discussion.
- “Resitting” the discussion will result in a “RESIT” for the whole course in Progress. Failing a “Resit discussion” will imply to fail the complete course.
- For each assignment question, you can add a formatted text cell to include (part of) your answer. Be sure to include everything you think it is necessary for the TA to understand your explanation during the discussion, e.g. references to tutorial exercises, references to specific formulas, computations, step-by-step small proves etc. Pointing at a formula with no further explanation is not enough of a justification for your answers.
- **Per discussion question, either 0% or 100% of points will be assigned.** Full points will be assigned only when meeting all these criteria:
 1. Both correctness and clearness in the presentation of the numerical results.
 2. Being able to explain the implemented code.
 3. Being able to explain **your** solutions to the theoretical exercises when requested.
 4. Being able to relate the theory to the obtained numerical results.
- Let Points_1 be the total number of points obtained in Lab 1, and let 20 be the total number of points available in Lab 1. Let also 10 be the maximum grade achievable in the course. The final grade Lab_1 is determined as

$$\text{Lab}_1 = 1 + \text{Code}_1 + 8 \frac{\text{Points}_1}{20} \in [1, 10],$$

where $\text{Code}_1 \in \{0, 1\}$ rewards good quality code, see instructions when starting the coding part.

Theoretical exercises

A Lagrange interpolation

We will consider the interpolation of some function f on the interval $[a, b]$.

Lagrange interpolation

1. Let $f(x) = \sin(x)$ on the interval $[0, 2\pi]$. Sketch f and the equidistant Lagrange interpolant of polynomial order 1, 2 and 3.
2. Give the definition of the order n Lagrange interpolant $\Pi_n f$ defined on the nodes $a = x_0, \dots, x_n = b$, in terms of the Lagrange basis functions

$$\varphi_k^{(n)}(x) = \prod_{\substack{j=0 \\ j \neq k}}^n \frac{x - x_j}{x_k - x_j}.$$

Show that it is an interpolant

$$\Pi_n f(x_i) = f(x_i), \quad i = 0, \dots, n,$$

and verify that this holds in your sketch.

3. The Lagrange interpolant can also be written in terms of the centered coefficients c_i

$$\Pi_n f(x) = \sum_{i=0}^n c_i (x - m)^{n-i}, \quad (1)$$

where $m = (a + b)/2$. Find c_i in terms of the values of $f(x)$ at the interpolatory nodes for $n = 1$ and $n = 2$.

4. Show that equidistant Lagrange interpolation of order n satisfies the following error bound

$$|E_n f(x)| = |f(x) - \Pi_n f(x)| \leq \frac{\max_{y \in [a, b]} |f^{(n+1)}(y)|}{4(n+1)} \left(\frac{b-a}{n} \right)^{n+1}, \quad (2)$$

by following the following steps:

- (a) Show that for equidistant nodes x_i , using the transformation $x = a + (b-a)s/n$, it holds that

$$|\omega_{n+1}(x)| \leq \left(\frac{b-a}{n} \right)^{n+1} \max_{s \in (0, n)} \prod_{j=0}^n |s - j|$$

for $x \in (a, b)$

- (b) Using the fact that:

$$\prod_{j=0}^n |s - j| \leq \frac{1}{4} n!$$

and the error formula from the lecture, show the statement in Equation (2).

B Piecewise interpolation and stability analysis

Accuracy of piecewise Lagrange interpolation

We denote by $\Pi_n^N f$ the piecewise Lagrange interpolant of order n using equidistant subintervals of size $H = (b-a)/N$, where N is the number of subintervals.

5. Sketch the piecewise Lagrange interpolant of order 1 for $f(x) = \sin(x)$ on the interval $[0, 2\pi]$ using four equidistant subintervals.

6. Sketch the piecewise Lagrange interpolant of order 2 using two equidistant subintervals.
7. Show that the piecewise Lagrange interpolant of order n satisfies the following upper bound (use Equation (2))

$$|E_n^N f(x)| = |\Pi_n^N f(x) - f(x)| \leq \frac{\max_{y \in [a,b]} |f^{(n+1)}(y)|}{4(n+1)} \left(\frac{b-a}{Nn} \right)^{n+1}. \quad (3)$$

8. How many times the function f has to be evaluated for computing the coefficients of the Lagrange interpolation of order n ? Why?
9. How many times the function f has to be evaluated for computing the coefficients of the piecewise Lagrange interpolation of order n , using N subintervals?

Stability analysis

Here we consider the situation where we interpolate perturbed function values $\hat{f}(x_i) = f(x_i) + \epsilon(x_i)$, for some perturbation $\epsilon(x)$. Such a perturbation can come from e.g. measurement errors, round-off errors, approximation errors or a combination thereof.

1. Let the Lebesgue constant Λ_n be given by

$$\Lambda_n = \max_{x \in [a,b]} \sum_{i=0}^n |\varphi_i^{(n)}(x)|.$$

Show that the following inequality holds

$$|\Pi_n f(x) - \Pi_n \hat{f}(x)| \leq \Lambda_n \max_{x \in [a,b]} |\epsilon(x)|,$$

where $\Pi_n \hat{f}$ interpolates the perturbed function values $\hat{f}(x_i)$. It can be shown that for equidistant interpolatory nodes the Lebesgue constant grows ‘almost exponentially fast’ in n

$$\Lambda_n \simeq \frac{2^{n+1}}{en \log n + \gamma}, \quad e \simeq 2.7183, \quad \gamma \simeq 0.5772.$$

2. Show that the total error satisfies the following upper bound (use Equation (2))

$$|\Pi_n \hat{f}(x) - f(x)| \leq \frac{\max_{x \in [a,b]} |f^{(n+1)}(x)|}{4(n+1)} \left(\frac{b-a}{n} \right)^{n+1} + \Lambda_n \max_{x \in [a,b]} |\epsilon(x)|. \quad (4)$$

Hint: use the triangle inequality.

3. Show that for piecewise interpolation we similarly find (use Equation (3))

$$|\Pi_n^N \hat{f}(x) - f(x)| \leq \frac{\max_{x \in [a,b]} |f^{(n+1)}(x)|}{4(n+1)} \left(\frac{b-a}{Nn} \right)^{n+1} + \Lambda_n \max_{x \in [a,b]} |\epsilon(x)|. \quad (5)$$

C Newton-Cotes quadrature

A piecewise Lagrange interpolant $\Pi_n^N f$ of f can be used to approximately compute the integral of f over its interpolation interval

$$\int_a^b f(x)dx \approx \int_a^b \Pi_n^N f(x)dx.$$

1. Use your previously made sketches to show how (piecewise) interpolation can be used for approximate integration.
2. Show that the antiderivative of the Lagrange polynomial $\Pi_n f$ (see (1)) is given by

$$\int \Pi_n f dx = \sum_{i=0}^n \frac{c_i}{n+1-i} (x-m)^{n+1-i}. \quad (6)$$

3. What is the degree of exactness of a numerical integration method? How does the degree of exactness relate to the order used in the polynomial approximation used to develop the integration method?
4. The Trapezoidal formula is given by

$$\int_a^b f dx \approx \frac{b-a}{2} (f(a) + f(b)).$$

- (a) Show that exact integration of the equidistant Lagrange interpolating polynomial of order 1 yields the Trapezoidal formula (use your answer to Question A.3 for $n = 1$).
- (b) The error using the Trapezoidal formula is given by

$$E_1(f) = -\frac{(b-a)^3}{12} f^{(2)}(\xi), \quad \xi \in [a, b].$$

Show that the degree of exactness of the Trapezoidal formula is 1.

- (c) Show that the composite Trapezoidal formula is second-order accurate in H

$$|E_1^H(f)| \leq H^2 \frac{(b-a) \max_{y \in [a,b]} |f^{(2)}(y)|}{12}.$$

5. Simpson's formula is given by

$$\int_a^b f dx \approx \frac{b-a}{6} (f(a) + 4f(m) + f(b)).$$

- (a) Show that exact integration of the equidistant Lagrange interpolating polynomial of order 2 yields Simpson's formula (use your answer to Question A.3 for $n = 2$).
- (b) The error using Simpson's formula is given by

$$E_2(f) = -\frac{(b-a)^5}{2880} f^{(4)}(\xi), \quad \xi \in [a, b].$$

Show that the degree of exactness of Simpson's formula is 3.

(c) Show that the composite Simpson formula is fourth-order accurate in H

$$|E_2^H(f)| \leq H^4 \frac{(b-a) \max_{y \in [a,b]} |f^{(4)}(y)|}{2880}.$$

6. Show that

$$\int_0^1 \sin^2(x) dx = \frac{1 - \cos(1) \sin(1)}{2}.$$

This will be useful for computing the integration error in the Lab.

D Gaussian quadrature

A Gauss-Legendre quadrature formula corresponds to an approximation of integrals of the type:

$$\int_{-1}^1 f(x) dx \approx \tilde{I}_m^G(f) = \sum_{k=1}^m \omega_k f(x_k) \quad (7)$$

with $x_1, \dots, x_m \in (-1, 1)$ the roots of the Legendre polynomial of degree m , and $\omega_1, \dots, \omega_m$ some weights. The derivation and some of the properties of can be found in the lecture (notes).

1. Prove that that $\omega_1, \dots, \omega_m > 0$. *Hint:* you could use the properties of the function $f(x) := ((x - x_i)^{-1} w_m(x))^2$, where $w_m \in \mathbb{P}_m$ is the Newton polynomial of degree m at the quadrature nodes.
2. Given the Gauss-Legendre formula (7), compute the quadrature rule $I_{m,[a,b]}^G(f)$ that allows to approximate an integral on an arbitrary interval $[a, b]$.
3. Show that the error of the Gauss-Legendre quadrature on an interval $[a, b]$ corresponds to:

$$E_{m,[a,b]}^G(f) := \left| \int_a^b f(y) dy - I_{m,[a,b]}^G(f) \right| \leq \frac{\max_{y \in [a,b]} |f^{(2m)}(y)|}{4m(2m-1)^{2m}} (b-a)^{2m+1}$$

Hint: You may want to use standard Lagrange interpolation to introduce auxiliary quantities, and “add zero” at some point in the proof. Also, you may need to use some of the results within this tutorial.

4. Determine the convergence order of a **composite** Gauss-Legendre quadrature with respect to H .

Computational exercises & discussion questions

Instructions for coding

- A python notebook with a skeleton code is given to you as part of the material of this unit. You can upload it in your Google Drive and work on the assignment using Google Colab or you can edit it locally in your laptop (please be aware that no instructions on the installation of python and/or on the setup of any code editor will be given to you during this course).

- Do not forget to import all the modules you need to make your code work at the very beginning of the notebook (e.g. `numpy` etc.). Try not to overwrite variables to avoid inconsistency: in case of sudden “nonsense” outputs, the first thing to try is restarting the runtime and run all again, then you proceed with the debugging phase. Coding simple test cases to check on each of your functions is another helpful tip for debugging your code.
- The “Code quality point” will be assigned if the following criteria are met for **all your answers**:
 1. Descriptive variable and function names make the code easy to read and understand.
 2. Functions clearly structure and organize the code, encapsulating tasks/code segments to improve readability and proneness to errors and to enable code reusability, and break up large sections (so-called “spaghetti code”).
 3. The code is well annotated with comments*.
 4. Avoid inconsistencies in the naming of functions and variables (choose a format and stick to it, e.g., `min_max_search()` vs. `minMaxSearch()`), avoid mixing single and double quotes for strings, and inconsistent indenting of code blocks.
 5. Functions MUST NOT use global variables!
 6. Output formatting: printed output should be easy to read and understand, e.g.:

```
Computing time of this function is: 10.3 secs
```

or

```
N = 10, error = 0.0012343234
```

Plots need to appear with labels and legends.

A Lagrange interpolation

Implement `lagrangeInterp` which returns the centered coefficients c_i of the equidistant Lagrange interpolating polynomial $\Pi_n f$ in (1) for a given function f , an interval $[a, b]$ and polynomial order $n > 0$. For the computation of the centered coefficients you may use numpy’s built-in `polyfit` by subtracting m to the values of the nodes $\mathbf{x}$ when calling the function. Verify that the values of the centered coefficients are the same you computed by hand in the tutorial for $n = 1$ and $n = 2$.

```

## Lagrange interpolation
#
# INPUT:
# f          scalar-valued function
# interval   interpolation interval [a,b]
# n          interpolation order
#
# OUTPUT:
# coeff      centered coefficients of Lagrange interpolant

def lagrangeInterp(f, interval, n):

```

*Helpful comments are ones that help explain code segments the meaning of which is not immediately clear (e.g., from good variable/function naming), or why (as opposed to how) something has to be done. Bad comments are those that state the obvious (e.g., the comment `#loop` before a `for` loop or `#check if x is greater than y` before `if x > y`).

Consider $f(x) = \sin(\frac{x}{p})$ on the interval $[0, 2p\pi]$, with $p \in \mathbb{N}$. You are given the code of a Python function `evaluateInterpolant` which evaluates a single (or piecewise) Lagrange interpolant. In a single figure plot the function f and the interpolating polynomials $\Pi_1 f(x), \dots, \Pi_4 f(x)$ on the given interval. The TA will set a value of p during the discussion, so make your whole code dependent on p . Then make a new figure, and using `semilogy` plot the error of the interpolating polynomials $|f(x) - \Pi_1 f(x)|, \dots, |f(x) - \Pi_4 f(x)|$ as a function of x .

Discussion question A.1: (2 point(s), see also Tutorial: A.2)

For each value of n , at how many points is the error between interpolant and function (very close to) zero in the plots? How can this number of points be explained from the theory?

For $\Pi_1 f, \dots, \Pi_{15} f$ compute the maximum error on the given interval, and plot that maximum error in the y-axis in terms of the polynomial order n in the x-axis using `loglog`. Also include in the plot the right-hand-side of Equation (2).

Discussion question A.2: (2 point(s), see also Tutorial: A.4)

Which of both curves (true error and right-hand-side of Equation (2)) is larger? How can this be explained from the theory?

B Piecewise interpolation and stability analysis

Piecewise Lagrange interpolation

Implement a Python function `piecewiseInterp` which computes the centered coefficients (centered per subinterval) of the piecewise Lagrange interpolant $\Pi_n^N f$, together with the corresponding subintervals for a given function f , an interval $[a, b]$, a specified polynomial order n and the number of subintervals N . For $N = 1$ the coefficient output has to be equivalent to that of `lagrangeInterp`.

```
## Piecewise interpolation
#
# INPUT:
# f                f scalar-valued function
# interval         interpolation interval [a, b]
# n               interpolation order
# N               number of subintervals
#
# OUTPUT:
# coeffs          i-th row are centered coefficients of i-th subinterval
# intervals       i-th row are begin and endpoint of i-th subinterval

def piecewiseInterp(f, interval, n, N):
```

Consider f as before and keep the code depending on p . In a single figure, plot the function f and the piecewise Lagrange interpolants of order 1, 2, 3 and 4 using $N = 4$ subintervals. Make another figure with the logarithm of the error as a function of x , for the piecewise interpolants with $n = 1, \dots, 4$ at a fixed number of subintervals $N = 4$.

For each order $n = 1, \dots, 4$, plot the maximum error over the whole interval v/s the number of subintervals $N = 2^l$, for $l = 0, \dots, 10$ using `loglog`, together with the right-hand side of Equation (3).

Discussion question B.1: (2 point(s), see also Tutorial: B.7)

For fixed n , each of the error curves $E_n^N f$ should result in (approximately) a straight line as a function of N in the double logarithmic plot. Explain that in view of the theory, and in particular why the slope of that lines changes with changing n . Show the error curves for $n = 1, 2, 3, 4$ in the same plot.

Comparison

Thus far we have seen two methods for increasing the interpolation accuracy: increasing the order n , or increasing the number of subintervals N . A fair comparison between those methods considers the maximum error as function of the number of function evaluations needed. In a single figure, using `loglog`, plot the following computed errors v/s the number of required function evaluations:

- One curve for the maximum (global) Lagrange interpolation error

$$\max_{x \in [a,b]} |E_n f(x)|,$$

where $n = 1, \dots, 20$ is varied in order to obtain different number of functions evaluations.

- One curve for each $n = 1, \dots, 4$: the maximum piecewise Lagrange interpolation error

$$\max_{x \in [a,b]} |E_n^N f(x)|,$$

where $N = 2^l$, $l = 0, \dots, 10$, is varied in order to obtain different number of functions evaluations.

Discussion question B.2: (3 point(s), see also Tutorial: A.4, B.7, B.9)

Which method of increasing the interpolation accuracy leads to a smaller number of function evaluations for a given accuracy? Justify your answer using the error estimation (3).

Consider now the interpolation of the perturbed function $\hat{f}(x) = f(x) + 10^{-3} \cos(\frac{8x}{p})$, $p \in \mathbb{N}$. Note that the total error is now given by $\Pi_n \hat{f}(x) - f(x)$ and $\Pi_n^N \hat{f}(x) - f(x)$ for Lagrange and piecewise Lagrange interpolation respectively. Hence you should use (4) and (5) as the corresponding upper bounds.

Create separate plots with:

- $f(x)$ and $\Pi_n \hat{f}(x)$ for $n = \{4, 20\}$.
- $f(x)$ and $\Pi_n^N \hat{f}(x)$ for $n = 1$, $N = \{4, 20\}$.
- $\max_x (\Pi_n \hat{f}(x) - f(x))$ v/s n , with $n = 1, \dots, 20$. Include also in the plot the right-hand-side of (4). Use `loglog`.
- $\max_x (\Pi_n^N \hat{f}(x) - f(x))$ v/s N , for $n = 1, 2, 3, 4$ with $N = 2^j$, $j=0, \dots, 10$. Include also in the plot the right-hand-side of (5). Use `loglog`.

Discussion question B.3: (3 point(s), see also Tutorial: B.1, B.2, B.3)

- Describe the behavior of $\max_x (\Pi_n \hat{f}(x) - f(x))$ with respect to n and explain it in view of the theory.

- Describe the behavior of $\max_x (\Pi_n^N \hat{f}(x) - f(x))$ with respect to N and explain it in view of the theory.
- In case that you would know that the “measured” function values are perturbed, but you just know that the perturbations correspond to random variables at each node every time, and therefore they will change every time you measure them, which combination of N and n would (most likely) lead to the a more similar error when repeating the measurements? Justify your answer in view of the theory.

C Newton-Cotes quadrature

We will now apply the previously developed interpolation functions to the approximate computation of an integral.

Composite integration

Given the centered coefficients c_i obtained from `piecewiseInterp` we can use (6) for the exact integration of the interpolating polynomial. Implement `compositeIntegr` which approximately computes the integral of some input function f using piecewise Lagrange interpolation of order n on N subintervals.

```
## Composite integration
#
# INPUT
# f                f scalar-valued function
# interval          interpolation interval [a, b]
# n                interpolation order
# N                number of subintervals
#
# OUTPUT
# num_integral      approximate integral

def compositeIntegr(f, interval, n, N):
```

Discussion question C.1: (2 point(s), see also Tutorial: C.4, C.5)

Give a few *numerical examples* using the function `compositeIntegr` you have just implemented to show that the Trapezoidal and Simpson’s rules have degree of exactness 1 and 3 respectively. Justify your answer referring to the theory.

Discussion question C.2: (3 point(s), see also Tutorial: C.4, C.5)

- For each $n = 1, \dots, 5$: compute the approximate integral of $f(x) = \sin^2(x)$ over the interval $[0, 1]$ using $N = 2^0, \dots, 2^8$ intervals, and compute the error for each N . Plot using `loglog` the error as function of the number of subintervals for each $n = 1, \dots, 5$.
- What could be the degree of exactness for each of the plotted values of n based on the error curves you just computed. How could you justify your answer from the theory? How could you test numerically if your hypothesis is correct?

D Gaussian quadrature

Implement a function `gaussLegQuad` that receive the number of quadrature points and returns the nodes and weights within the interval $[-1, 1]$ of the Gauss-Legendre quadrature.

Then, implement a function `compositeIntegralGaussLeg` that computes the composite Gauss-Legendre integration for a function f on an arbitrary interval $[a, b]$.

Discussion question D.1: (*3 point(s), see also Tutorial: D.3, D.4*)

Compute convergence curves – analogous to the previous discussion question – for $m = 1, 2, 3, 4$ and determine for each case the convergence order empirically. How does the empirically one compared to the one you computed in the tutorial? Present also your on-paper computation the convergence order. What quadrature formula would you prefer (Gauss-Legendre or Newton-Cotes), and why?

Please give us your feedback during and/or right after the discussion with the TA. **THIS IS THE LINK TO THE FEEDBACK FORM.**